Módulo 2 - Budín de Banana

Hilario Cavalli, Lucía Dominguez, Milagros Farina, María Romero y Julieta Valle

Abril 2026

Algoritmos

01/04/2026

R Markdown

Este es un documento de R Markdown. Markdown es una sintaxis de formato simple para la creación de documentos HTML, PDF y MS Word. Para más detalles sobre el uso de R Markdown, consulta http://rmarkdown.rstudio.com.

Cuando hagas clic en el botón Knit (Tejer), se generará un documento que incluirá tanto el contenido como la salida de cualquier bloque de código de R (chunks) incrustado en el documento.

Se puede agregar el parámetro echo = FALSE al bloque de código (chunk) para evitar que se muestre el código de R. De esta manera, en el documento final solo se visualizará el resultado (por ejemplo, un gráfico) y no las instrucciones técnicas que lo crearon.

Primeros comandos de R

R es un lenguaje muy parecido a Mathlab y trabaja con variables que generalmente son matrices.

```
 a=38
 a=40
 b=60
c=b-a
c
```

```
## [1] 20
```

```
y=40
alfa<-30
```

Uso de datasets o base de datos internos de R

Al ejecutar el comando data() en la consola, R nos muestra los diversos conjuntos de datos precargados que tenemos disponibles para trabajar. Una vez que elegimos una tabla, podemos acceder a una columna específica utilizando el operador $. Al escribir el nombre del dataset seguido de este símbolo, R nos permite seleccionar la variable deseada para analizar sus datos de manera individual sin necesidad de procesar toda la tabla completa.

```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
```

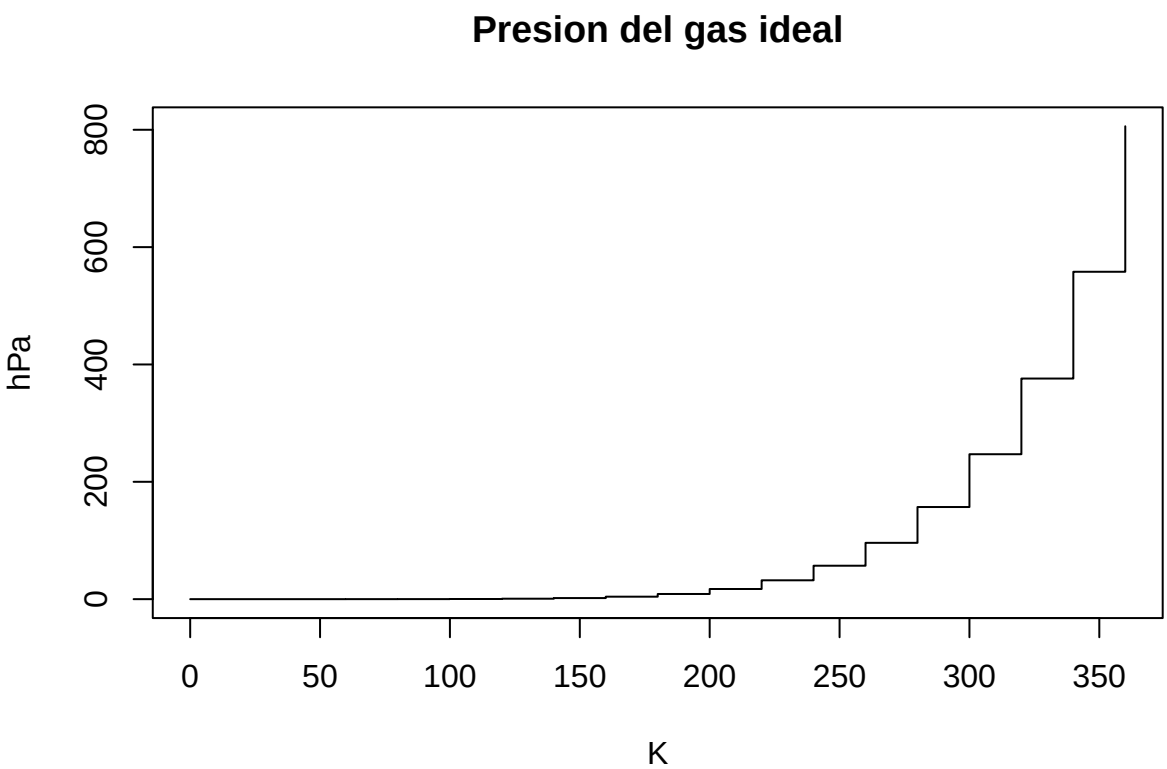
```
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando Plot

Es una función que permite graficar diversas fuentes de datos. Además, podemos asignar nombres a los ejes, títulos y personalizar la estética del gráfico (colores, tipos de línea, etc.). En caso de dudas sobre su sintaxis o argumentos, podemos escribir ?plot en la consola, lo cual abrirá una ventana de ayuda con la documentación detallada y ejemplos de uso del comando.

Por ejemplo:

```r
plot(pressure, main="Presion del gas ideal", ylab="hPa", xlab="K", type="s")
```



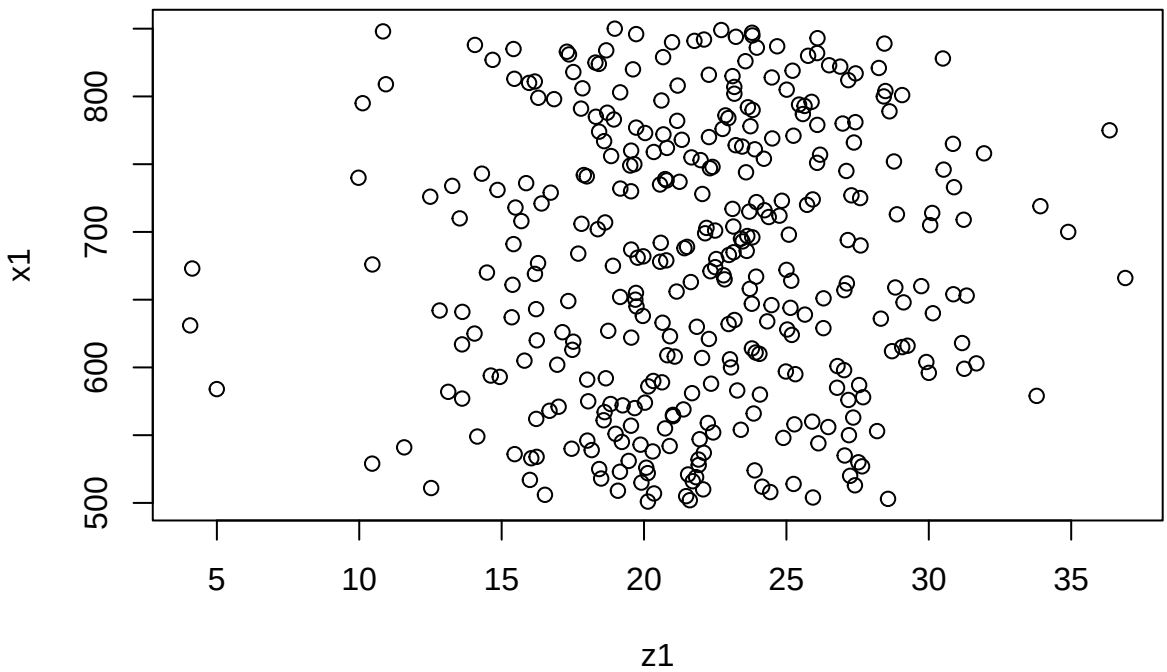
Funciones estadisticas

```r
z1<-rnorm(350,22,5)
z1
```

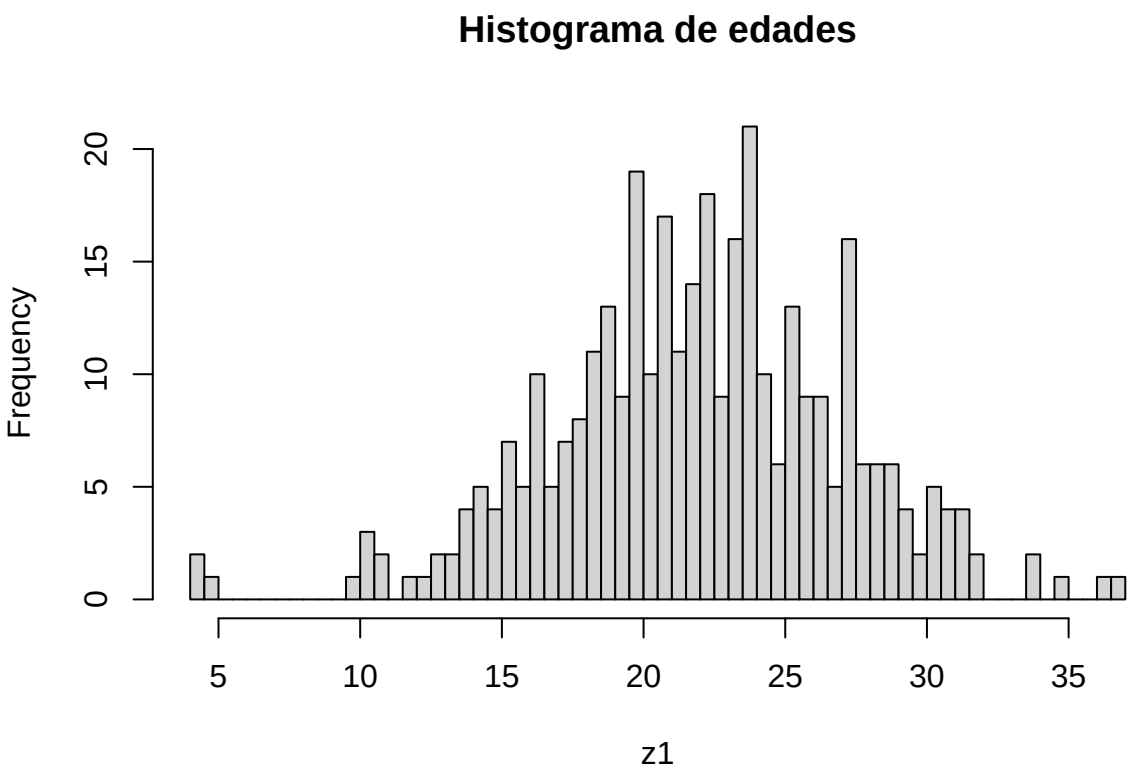
```
##   [1] 20.136317 21.607845 28.567610 25.931782 21.479291 16.520468 20.350420
##   [8] 24.433473 19.082644 22.076421 12.525254 24.148848 27.406103 25.252284
##  [15] 19.909078 21.711148 15.993060 18.494600 21.828077 27.237475 21.547627
##  [22] 20.131133 19.154674 23.884935 18.425977 20.079243 27.656125 21.920468
##  [29] 10.453427 27.527908 19.461338 21.915017 16.040711 16.227321 27.045954
##  [36] 15.456286 22.098814 20.304549 18.162667 17.457280 11.578205 20.900000
##  [43] 19.878072 26.122414 19.222784 18.009187 21.953211 24.891939 14.144723
##  [50] 27.195662 19.000222 22.429145 28.183649 23.399034 20.736790 26.469596
##  [57] 19.541282 25.280639 22.239252 25.906749 18.581521 16.214410 27.344908
##  [64] 21.018097 21.016000 23.850245 18.616294 16.680835 21.383427 19.671189
```

```
##  [71] 17.001114 19.249284 18.826212 20.031873 18.039592 27.177241 13.612922
##  [78] 27.692465 33.787867 24.070506 21.687054 13.124230 23.272417  4.996903
##  [85] 26.778712 20.151645 27.553407 22.353919 20.632084 20.325497 18.003772
##  [92] 18.658886 14.933944 14.619459 25.312886 30.000561 24.980570 27.025548
##  [99] 31.245062 23.053951 26.793064 16.952499 31.671258 29.913855 15.801158
## [106] 23.017257 22.037911 21.078777 20.819279 24.051353 23.918858 28.708399
## [113] 17.482784 23.789251 29.065350 29.252889 13.609856 31.169275 17.515885
## [120] 16.234366 22.278661 19.551503 20.910806 25.188645 14.047434 17.135273
## [127] 18.742630 25.029899 26.299329 21.853788  4.062447 22.970814 20.650664
## [134] 24.327127 23.175742 28.316343 15.354445 19.957183 25.649416 30.142682
## [141] 13.620651 12.822578 16.209630 25.142837 19.735915 24.475440 23.785108
## [148] 29.113188 17.340287 19.695613 26.303461 19.161554 31.329209 30.861181
## [155] 19.717882 21.144002 27.034554 23.714017 28.823103 29.735661 15.382559
## [162] 27.122929 21.643794 25.170371 22.829755 36.899456 23.935637 22.792575
## [169] 16.169642 14.483134 22.333516 25.003178  4.134935 22.497956 18.907611
## [176] 10.460398 16.277992 20.565385 20.781383 22.537437 19.776216 19.975185
## [183] 22.978817 17.690303 23.145069 23.605602 19.547925 21.418668 21.510333
## [190] 27.604436 15.416464 20.587678 23.458478 27.155341 23.404281 23.803389
## [197] 23.626426 25.086210 22.144498 34.895518 22.496731 18.376339 22.193288
## [204] 23.136945 30.047433 17.803813 18.633018 15.693339 31.224695 13.527413
## [211] 24.379394 24.755523 28.879752 30.119681 23.697380 24.230618 23.107962
## [218] 15.485223 33.918607 25.724719 16.401806 23.947094 24.841221 25.924109
## [225] 27.589140 12.493307 27.283191 22.048004 16.722812 19.544991 14.855387
## [232] 19.168373 30.885018 13.265034 20.562805 15.860840 21.245807 20.793050
## [239] 20.740952  9.972692 17.989416 17.886266 14.306971 23.581607 27.105319
## [246] 30.520036 22.314599 22.403916 19.515540 19.659617 26.077809 28.776322
## [253] 21.986311 24.208891 21.665564 18.844804 26.182156 31.938971 20.343269
## [260] 19.547469 23.890820 20.803573 23.447224 23.219449 30.848112 27.370007
## [267] 18.599062 21.326914 24.502768 22.281040 25.247270 20.681633 20.038126
## [274] 18.422247 36.348377 22.759736 19.724240 23.736924 26.084311 26.974895
## [281] 27.421359 21.163743 18.940467 22.960990 18.309028 22.858555 25.578313
## [288] 18.707968 28.619152 23.812226 17.788155 23.647349 25.637011 25.444309
## [295] 10.120172 25.883737 20.612043 16.841160 16.287802 28.419828 29.063905
## [302] 23.172172 19.162663 28.475885 25.010030 17.838834 23.169591 21.181287
## [309] 10.934745 15.962701 16.163616 27.173067 15.445611 24.486204 23.106827
## [316] 22.275595 27.430174 17.517250 25.215532 19.611603 28.245178 26.890098
## [323] 26.500387 18.413402 18.293219 23.560833 14.683323 30.494547 20.669053
## [330] 25.761303 17.364798 26.077741 17.290929 18.672642 15.417000 23.961315
## [337] 24.675191 14.062086 28.441854 20.984172 21.770321 22.105348 26.091491
## [344] 23.225406 23.806016 19.723393 23.793123 10.834225 22.713203 18.972352
```

```r
w1<-length(z1)
x1<-(501:850)
plot(z1,x1)
```




```r
hist(z1,main="Histograma de edades",breaks=50)
```

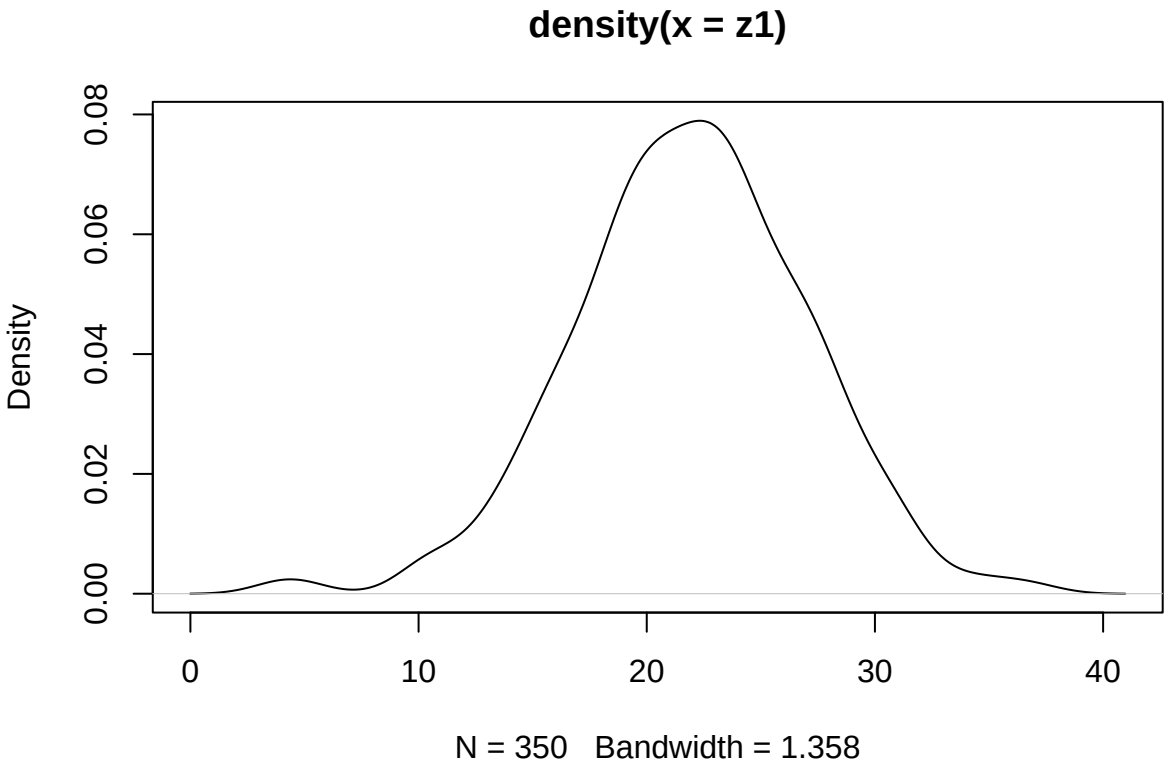



```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.358
```

```
##
##        x                  y
##  Min.   :-0.01016   Min.   :1.203e-05
##  1st Qu.:10.23539   1st Qu.:1.846e-03
##  Median :20.48095   Median :9.475e-03
##  Mean   :20.48095   Mean   :2.435e-02
##  3rd Qu.:30.72651   3rd Qu.:4.628e-02
##  Max.   :40.97207   Max.   :7.895e-02
```

```r
plot(density(z1))
```



Ejercicios Algoritmos

08/04/2026

Ejercicio 1

Consigna: Las últimas tres cifras del DNI son 112, crear una variable que tenga esos números.

```
DNI<-112
```

Consigna: Crear un vector que tenga todos los números enteros desde el 1 al 112

```
secuencia_dni<-(1:112)
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112
```

Ejercicio 2

Consigna: Calcular la suma de todos los valores del vector secuencia_dni

```
total<-0
n<-length(secuencia_dni)
for (i in 1:n)
  total<-total+i
print("La suma calculada en R es:")
```

```
## [1] "La suma calculada en R es:"
```

```
  total
```

```
## [1] 6328
```

Ejercicio 3

Repetir el ejercicio anterior en Python

```
suma_total = 0

# Usamos 113 porque el limite superior en range() no se incluye
for i in range(1, 113):
    suma_total += i

print(f"La suma calculada en Python es: {suma_total}")
```

```
## La suma calculada en Python es: 6328
```

Ejecicio 4

Consigna: ¿Cuánto tarda en correr el código del ejercicio 2?

```r
inicio<-Sys.time()
total<-0
valor_final<-1000000*length(secuencia_dni)
for(i in 1:valor_final) {
  total<-total+1
  total
}
final<-Sys.time()
final-inicio
```

```
## Time difference of 2.033944 secs
```

Ejercicio 5

Repetir la suma usando la cabeza de Gauss

```r
n <- 112
suma_gauss <- (n * (n + 1)) / 2

print("La suma usando la fórmula de Gauss es:")
```

```
## [1] "La suma usando la fórmula de Gauss es:"
```

```r
suma_gauss
```

```
## [1] 6328
```

Elementos de Programación

08/04/2026

Ejercicio 1.4: Generar un vector secuencia

Debemos generar un vector secuencia con for y con R, luego comparar la performance con systime

EXPERIMENTO A: Secuencia generada con bucle 'for'

Este método suele ser mas lento porque R tiene que reasignar memoria constantemente al ir aumentando el tamaño del vector.

```
tiempo_for <- system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})

# Mostrar extremos del vector resultante
print("Extremo Inicial (head):")
```

```
## [1] "Extremo Inicial (head):"
```

```
print(head(A))
```

```
## [1]  2  4  6  8 10 12
```

```
print("Extremo Final (tail):")
```

```
## [1] "Extremo Final (tail):"
```

```
print(tail(A))
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```
tiempo_seq <- system.time({
  # Generamos la secuencia de 2 en 2 hasta 100,000
  B <- seq(from = 2, to = 100000, by = 2)
})

# Mostrar extremos del vector resultante
print("Extremo Inicial (head):")
```

EXPERIMENTO B: Secuencia generada con comandos nativos de R (seq)

```
## [1] "Extremo Inicial (head):"
```

```
print(head(B))
```

```
## [1]  2  4  6  8 10 12
```

```
print("Extremo Final (tail):")
```

```
## [1] "Extremo Final (tail):"
```

```r
print(tail(B))
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

COMPARATIVA DE RESULTADOS

```r
cat("RESUMEN DE PERFORMANCE (Valores en segundos)\n")
```

```
## RESUMEN DE PERFORMANCE (Valores en segundos)
```

```r
cat("Tiempo con bucle FOR:\n")
```

```
## Tiempo con bucle FOR:
```

```r
print(tiempo_for)
```

```
##    user  system elapsed
##   0.011   0.004   0.015
```

```r
cat("Tiempo con funcion SEQ:\n")
```

```
## Tiempo con funcion SEQ:
```

```r
print(tiempo_seq)
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

Conclusión:

Al comparar los métodos para generar secuencias, se evidencia que la función nativa `seq` es significativamente más eficiente que el bucle `for`. Mientras que el bucle requiere una reasignación constante de memoria que eleva el tiempo de ejecución, la vectorización de R optimiza el uso del procesador.

Esta diferencia en los resultados obtenidos, como los valores de 0.011 y 0.001, confirma que el benchmarking es esencial para elegir el algoritmo más conveniente según el recurso físico disponible.

Ejercicio 1.5 y 1.6: Implementación de una serie Fibonachi

Para resolver la serie de Fibonacci en R, necesitamos usar un bucle que sume los dos valores anteriores para generar el siguiente, tal como indica la fórmula de recurrencia.

Consigna: ¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```r
# 1. Inicializamos la serie con los dos primeros valores
fib <- c(0, 1)
i <- 2  # Contador de elementos (iteraciones iniciales)

# 2. Bucle para generar la serie hasta superar el millon
# f(n+1) = f(n) + f(n-1)
while (fib[i] <= 1000000) {
    proximo_valor <- fib[i] + fib[i-1]
    fib <- c(fib, proximo_valor)
    i <- i + 1
}

# 3. Resultados
cat("El primer numero de la serie mayor a 1.000.000 es:", fib[i], "\n")
```

```
## El primer numero de la serie mayor a 1.000.000 es: 1346269
```

```r
cat("Se necesitaron", i, "elementos en la serie.\n")
```

```
## Se necesitaron 32 elementos en la serie.
```

```r
cat("La cantidad de iteraciones (calculos realizados) fue:", i - 2, "\n")
```

```
## La cantidad de iteraciones (calculos realizados) fue: 30
```

```r
# Mostramos los ultimos valores para verificar
print("Ultimos valores de la serie:")
```

```
## [1] "Ultimos valores de la serie:"
```

```r
print(tail(fib))
```

```
## [1]  121393  196418  317811  514229  832040 1346269
```

Conclusión:

Mientras que el vector final contiene 32 términos, los primeros dos valores no requieren procesamiento por ser condiciones iniciales del problema. Por lo tanto, el procesador ejecutó el ciclo de suma 30 veces para alcanzar el objetivo, demostrando la eficiencia del lenguaje para resolver progresiones matemáticas complejas en milisegundos.

Ejercicio 1.7: Ordenación de un vector por método burbuja

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

Lo primero es instalar la librería microbenchmark para poder correr el código.

```r
# 1. Definimos la función burbuja
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)){
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# 2. Generamos la muestra de tamaño 20.000
muestra_20k <- sample(1:100000, 20000)

# 3. Consigna: Comparar performance con microbenchmark
library(microbenchmark)
resultado_performance <- microbenchmark(
  "burbuja(manual)" = burbuja(muestra_20k),
  "sort(nativo R)"  = sort(muestra_20k),
  times = 1
)
```

```r
# Mostramos el resultado en segundos
print(resultado_performance, unit = "s")
```

```
## Unit: seconds
##             expr         min          lq        mean      median          uq
##  burbuja(manual) 20.72780198 20.72780198 20.72780198 20.72780198 20.72780198
##   sort(nativo R)  0.00115961  0.00115961  0.00115961  0.00115961  0.00115961
##          max neval
##  20.72780198     1
##   0.00115961     1
```

Conclusión:

Al comparar ambos métodos para ordenar una muestra de 20.000 elementos, se observa una brecha de eficiencia abismal entre el algoritmo de burbuja (manual) y la función sort (nativo R). Mientras que la implementación manual requiere aproximadamente 20 segundos debido a su lógica de intercambio directo y su costo computacional cuadrático, el método nativo resuelve la tarea en menos de un milisegundo.

Ejercicio 1.8: La penitencia de Newton

Consigna: Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×10^{10} y verifica cuál de los dos es más eficiente.

```r
library(tictoc)

n_objetivo <- 1e10

# MÉTODO 1: Algoritmo de Sumas Parciales (Bucle For)

# Este método representa la "penitencia" de Newton: sumar uno a uno.

n_demo <- 1e9

tic("Tiempo del Bucle")
suma_bucle <- 0
for(i in 1:n_demo) {
  suma_bucle <- suma_bucle + i
}
toc()
```

```
## Tiempo del Bucle: 16.526 sec elapsed
```

```r
# MÉTODO 2: Algoritmo de la Función Matemática (Gauss)

tic("Tiempo de la Fórmula")
# Usamos la fórmula: (n * (n + 1)) / 2
suma_formula <- (n_objetivo * (n_objetivo + 1)) / 2
toc()
```

```
## Tiempo de la Fórmula: 0.001 sec elapsed
```

```r
# RESULTADOS FINALES
cat("Resultado final de la suma de Newton:", suma_formula, "\n")
```

```
## Resultado final de la suma de Newton: 5e+19
```

Conclusión

El algoritmo iterativo representa la penitencia de procesar cada suma individualmente, mientras que la fórmula de Gauss permite obtener el resultado de forma inmediata.

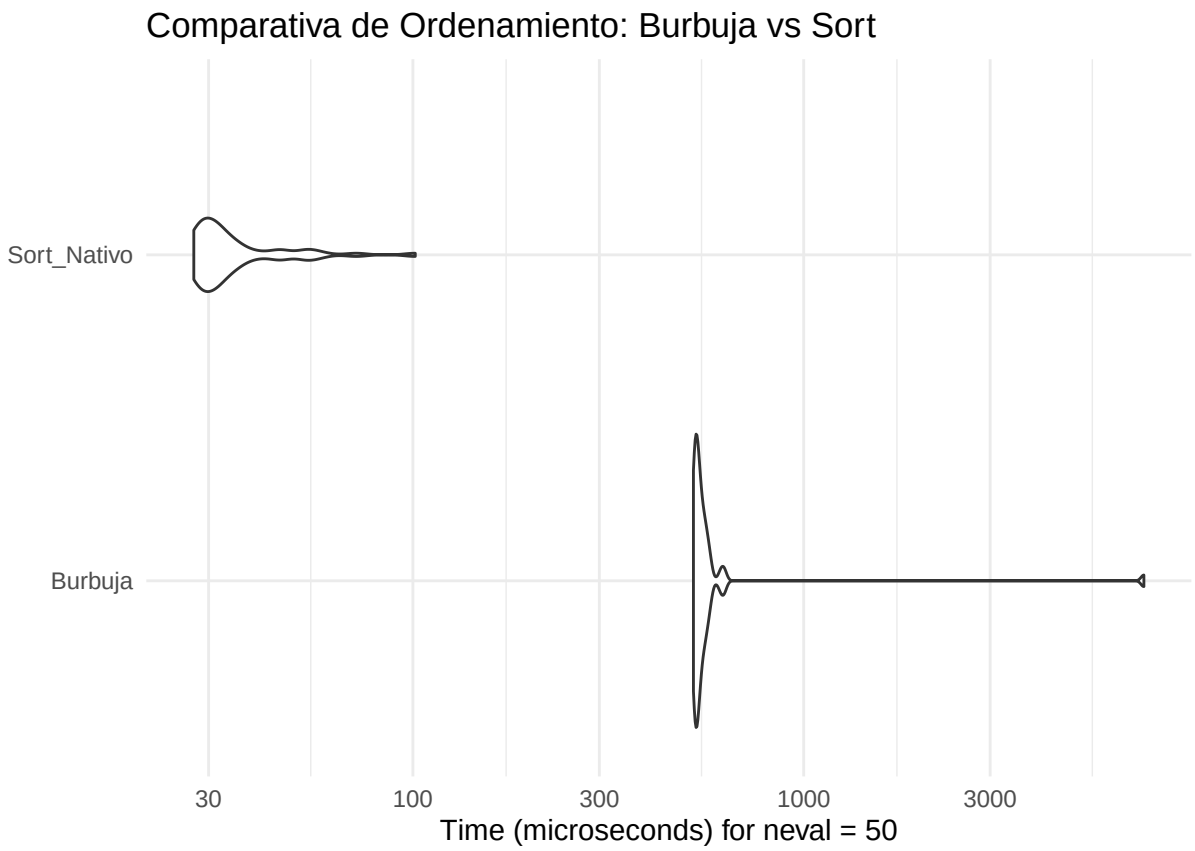
Para realizar un benchmarking efectivo sin comprometer la estabilidad del sistema, se utilizó un valor de n_demo = 1e9 en el primer método. Esta decisión técnica responde a que ejecutar un bucle hasta el objetivo real de 1×10^{10} demandaría horas de procesamiento y excedería la memoria RAM de Posit Cloud.

K-means y Gráficos de Violín

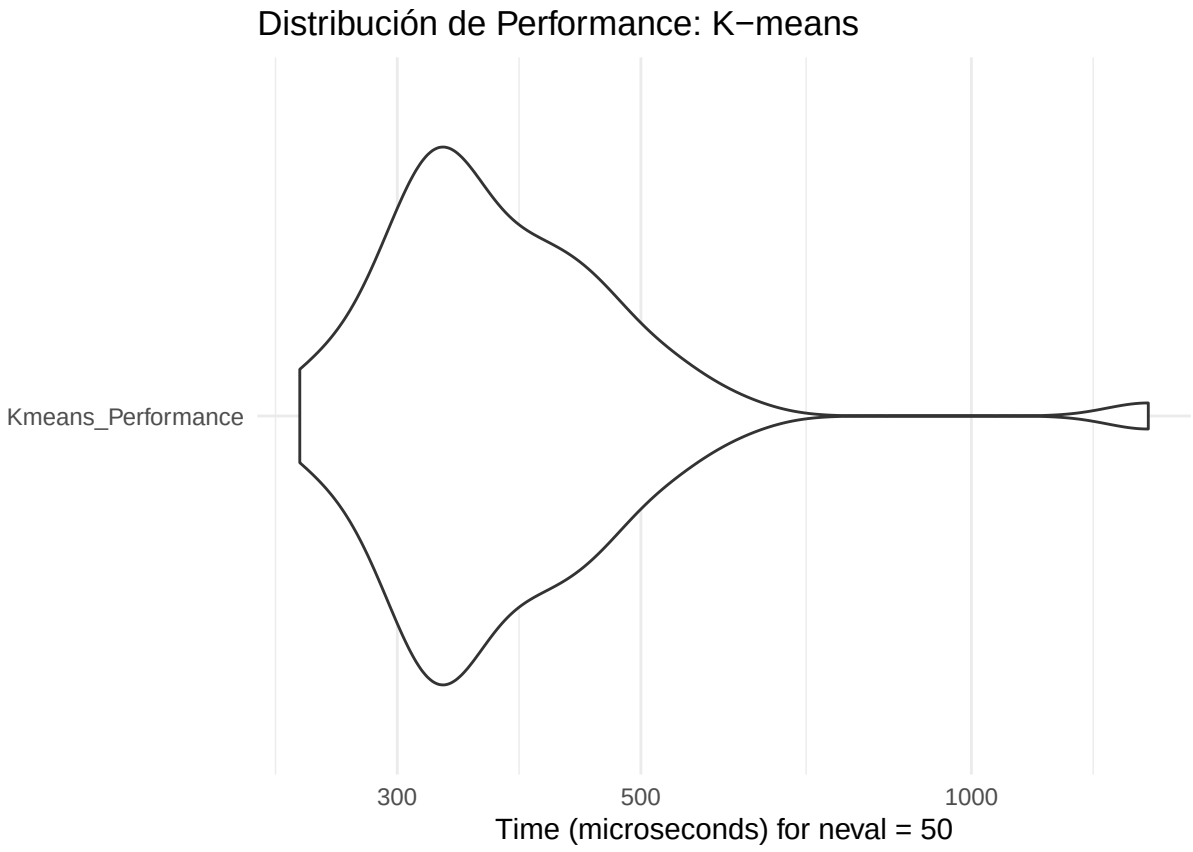
El algoritmo K-means es un método de agrupamiento que busca dividir un conjunto de datos en k grupos basados en la cercanía a un centroide. Por otro lado, un gráfico de violín es una combinación de un diagrama de caja y un gráfico de densidad; nos permite ver no solo el promedio de tiempo que tarda un código, sino también su variabilidad.

```r
library(microbenchmark)
library(ggplot2)

# 1. Definimos el método Burbuja (Bubble Sort)
burbuja <- function(x) {
  n <- length(x)
  for (i in 1:(n - 1)) {
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# 2. Creamos datos de prueba
datos_sort <- runif(100) # 100 números aleatorios
datos_kmeans <- matrix(runif(2000), ncol = 2) # 1000 puntos para K-means

# 3. COMPARATIVA: Burbuja vs Sort Nativo
benchmark_sort <- microbenchmark(
  Burbuja = burbuja(datos_sort),
  Sort_Nativo = sort(datos_sort),
  times = 50
)

# 4. MEDICIÓN: K-means
benchmark_kmeans <- microbenchmark(
  Kmeans_Performance = kmeans(datos_kmeans, centers = 3),
  times = 50
)

# 5. Generar Gráficos de Violín
# Gráfico de Ordenamiento
autoplot(benchmark_sort) +
  ggtitle("Comparativa de Ordenamiento: Burbuja vs Sort") +
```

```r
  theme_minimal()
```




```r
# Gráfico de K-means
autoplot(benchmark_kmeans) +
  ggtitle("Distribución de Performance: K-means") +
  theme_minimal()
```



Conclusión

La implementación de K-means demuestra que la eficiencia en el agrupamiento depende directamente de la naturaleza estocástica del algoritmo. Los gráficos de violín revelan una variabilidad en los tiempos de ejecución vinculada a la posición inicial de los centroides, lo que confirma que el rendimiento no es constante y requiere de múltiples inicios para asegurar estabilidad y convergencia.